

The Cortex-M port

How `micro-os-plus-iii-cortexm` is built, what its four boards are, and where its SMP lives.

This is the third architecture project, beside `-aarch32` and `-aarch64`. It is built the same way and from the same kernel, and it is the first port here with boards on **both** sides of the SMP line: three at `OS_NCPU=1` and one at `OS_NCPU=2`.

1. What it is

```
micro-os-plus-iii-cortexm/
├─ CMakeLists.txt
├─ include/cmsis-plus/rtos/port/      upstream's port contract
├─ include-rp2350/.../port/          the same, plus an SMP branch
├─ src/rtos/os-core.cpp              upstream: PendSV, SysTick, criticals
├─ src/rtos/os-core-rp2350.cpp        the same, plus the SMP branch
├─ src/semihosting-exit.cpp          strong _Exit() through SYS_EXIT
├─ test/
  ├─ CMakeLists.txt  hw.sh  qemu.sh
  ├─ boards/<id>/      board.cmake, include/, src/, linker.ld,
  │                    openocd.cfg, hw.sh  (pico2 also: glue/)
  └─ <id>/              that board's test applications
```

board	part	CPUs	applications
<code>nucleof411</code>	STM32F411RE, Cortex-M4F	1	<code>mos-test1</code>
<code>weactf411</code>	STM32F411CE, Cortex-M4F	1	<code>mos-test1</code> , <code>spi-pipeline</code>
<code>weactf412</code>	STM32F412RE, Cortex-M4F	1	<code>mos-test1</code> , <code>uart-test1</code>
<code>pico2</code>	RP2350, 2× Cortex-M33	2	<code>smp-test1</code> ... <code>smp-test5</code> , <code>smp-mat-test</code>

All four are **hardware-only**: none sets `UOS_BOARD_LINKER_QEMU`, so each builds one image per test and `test/qemu.sh` answers by naming `hw.sh`.

2. Where the SMP lives — the board, not the port

The design spec framed step 4 as *merging* pico2's dual-core core with the STM32 boards. Reading pico2's core changed that, and the new shape is better:

pico2's SMP is RP2350 silicon.

kernel lock	SIO hardware spinlock 0 . Not LDREX/STREX: the RP2350 has no global exclusive monitor, so the architectural route does not work across the two cores.
IPI	the SIO inter-core FIFO on <code>SIO_IRQ_FIFO</code> (external IRQ 25). The handler drains the FIFO and pends its own core's PendSV.
CPU index	<code>SIO_CPUID</code> .

None of that is Cortex-M — it is one SoC's. In this layout that makes it board code, exactly as the Lyra's GIC-400 SGI and the Pi's are board code. So `cortexm` did not gain SMP by changing its ISA file; it gained SMP because a board arrived that supplies a lock and an IPI.

Core 1 is launched through the bootrom FIFO handshake with its own MSP. Only core 0 advances the RTOS clock; core 1's SysTick reschedules core 1 alone.

Proof it is really in the image

Symbols from the linked `smp-test2-hwd`, not from the build log:

```
T port_cpu_id                T SIO_IRQ_FIFO_Handler
T PendSV_Handler            T SysTick_Handler
D os::rtos::port::scheduler::_smp_klock
T multicore::launch_core1(void (*)(), unsigned long)
b os::rtos::port::scheduler::core1_msp_stack

B os::rtos::port::scheduler::lock_state      size 2
B os::rtos::scheduler::os_idle_thread_core  size 8
```

`lock_state` is `state_t[OS_NCPU]` and measures **2** bytes; `os_idle_thread_core` is `thread*[OS_NCPU]` and measures **8**. `OS_NCPU=2` reached the scheduler.

3. Two port cores, and why they are not one

`src/rtos/os-core.cpp` is upstream's, single-core. `src/rtos/os-core-rp2350.cpp` is the predecessor's pico2 core: **the same file plus an `OS_USE_SMP_SCHEDULER` branch**, 91% identical by line. Its port headers already handle every M profile — `__ARM_ARCH_6M__`, `7M`, `7EM`, `8M_MAIN` — and every RP2350 reference in them is inside the SMP branch.

So they *could* be one file. They are not, for a reason rather than an oversight: **the three STM32 boards are hardware-proven on upstream's core**. Moving them onto a different 748-line scheduler is a change

that has to be re-proven on the desk, not assumed — and the standing rule here is that adding a board must not reach boards already tested.

A board therefore names the core it was proven with:

```
set (UOS_BOARD_PORT_CORE      "src/rtos/os-core-rp2350.cpp")
set (UOS_BOARD_PORT_INCLUDE  "include-rp2350")
```

Defaults are upstream's, so a board that says nothing gets what the STM32 boards get. Merging the two is a later, explicit step.

`pico2` forces `-D__ARM_ARCH_7EM__` although the M33 is ARMv8-M, as the predecessor's Makefiles did. The `v7E-M` path is the one that board was brought up on; letting the compiler select `__ARM_ARCH_8M_MAIN__` would change the scheduler under a working board.

4. What else became a board choice

Two things the port used to hard-code, because the boards disagree:

	STM32F4	pico2
startup	<code>micro-os-plus::iii-startup</code> — upstream's generic startup	its own <code>boot.S</code> + <code>interrupt-vectors.S</code>
trace backend	<code>micro-os-plus::iii-trace-semihosting</code>	its own <code>trace-uart.cpp</code> , over the UART

Both are named in each board's `UOS_BOARD_LIBS`. This is why the kernel splits them out as separate targets in the first place.

5. `src/` and `glue/` on pico2

pico2 is the only board whose sources split in two, and the split is read off the predecessor's seven Makefiles, not invented:

<code>src/</code>	<code>boot.S</code> , <code>interrupt-vectors.S</code> , <code>init-fini-stubs.c</code> , <code>clocks.cpp</code> , <code>led.cpp</code> , <code>multicore.cpp</code> — every Makefile listed them
<code>glue/</code>	<code>uart.cpp</code> , <code>rtos-glue.cpp</code> , <code>trace-uart.cpp</code> , <code>syscalls.c</code> — not every Makefile listed them

Two tests replace one of the second group:

- `smp-test5` defines its own `uart::` in `main.cpp`, so it must not also get the board's `uart.cpp`.

- `smp-test0` has no RTOS at all, so no glue, no trace, no syscalls.

A glob over `src/` linked both copies and failed on duplicate definitions — `uart::init()`, `uart::put_char()`, `HardFault_Handler`. `test/pico2/tests.cmake` composes them per test through the `board_test_sources()` hook, which is the same hook the Lyra uses for the same reason.

6. Running them

OpenOCD only. No GDB anywhere in this path, and nothing redirected.

```
BOARD=nucleof411 ./test/hw.sh          # list what is built
BOARD=nucleof411 ./test/hw.sh mos-test1
BOARD=pico2      ./test/hw.sh smp-test2
```

The STM32 boards: program, `arm semihosting enable`, `reset run`. Semihosting must be enabled **before** the image runs — `os::trace::printf()` issues `BKPT 0xAB`, which faults if the debugger is not listening for it.

The Pico 2 is different in two ways, both learned on the board and both carried in its `hw.sh`:

- `reset init`, not `reset`. A bare reset does not re-run the RP2350 bootrom/XIP setup, so the freshly programmed image never boots.
- **Resume** `cm1` **before** `cm0`. With `USE_SMP 0` the rp2350 target exposes two targets and `reset init` halts both, while a single `resume` resumes only the current one. A debug-halted core 1 will **not** boot from the software PSM reset `launch_core1()` issues — the launch then hangs forever waiting for the bootrom readiness word. Resuming `cm1` first leaves it in the bootrom core-1 launch loop, where it answers the FIFO handshake.

pico2 traces over its own UART, not semihosting, so OpenOCD exits once both cores are running and the console is the board's serial port.

7. Strong `_Exit()`

The same behaviour as the two A-profile ports, with this ISA's trap. The kernel's `_Exit()` is weak, so `src/semihosting-exit.cpp` overrides it and ends the run through semihosting `SYS_EXIT` rather than idling.

Nothing about the trap is written in this port. The kernel already ships `cmsis-plus/arm/semihosting.h`, which selects `bkpt` from `__ARM_ARCH_7M__` / `7EM` / `6M`, so this file calls its `report_exception()`. There is no `semihosting.hpp` here: the AArch32 one exists because on A-profile the trap depends on which OpenOCD *target type* drives the core (`cortex_a` → `SVC`, `aarch64` → `HLT`), a choice that does not arise on Cortex-M.

Verified in the linked image:

```
080038f4 T _Exit          <- strong, overriding the kernel's weak one
      movs r4, #24        <- 0x18 = SYS_EXIT
      bkpt 0x00ab
      .word 0x00020026     <- ADP_Stopped_ApplicationExit
      .word 0x00020023     <- ADP_Stopped_RunTimeError
```

8. What is not done

- **Nothing on this port has been run on hardware.** The five STM32 applications and the six pico2 tests link. That is all that is claimed.
- `smp-test0`, parked in `test/pico2/.pending/`. It compiles no kernel — nine files, none of them $\mu\text{OS++}$ — because it is the bare-metal dual-core bring-up test that runs *before* any scheduler exists. `uos_add_app()` always links `micro-os-plus:iii`, so there is no way to express that today, and adding one is a change to the helper every architecture project shares. It is not to be rewritten as an RTOS test; its value is that it runs without one.
- **The other three pico2 trees.** `pico2-std` (Peterson's algorithm in software), `pico2-sdk` (the Pico SDK) and `pico2-sdk-min` (byte-identical port files to `pico2-sdk` — not a separate port). They differ from `pico2` in the kernel lock and nothing else, so each is a sibling board when wanted.
- **Merging the two port cores** (§3).
- **The rest of pico2's ~40 application directories** — XIP loaders, USB, PSRAM. None of them is about SMP.